**Krishna Dalvi**
**D20C/10**

# Blockchain : Experiment - 3

## Aim:

Create a Cryptocurrency using Python and perform mining in the Blockchain created.

## Theory:

1. **Blockchain Overview**
   Blockchain is a distributed and decentralized ledger that stores information in a series of linked blocks. Each block contains:
   a. Transaction data
   b. Timestamp
   c. Previous block's hash
   d. Its own unique hash (digital fingerprint)
   Once data is recorded in a blockchain, it becomes immutable because altering one block would require recalculating all subsequent blocks.
2. **Mining**
   Mining is the process of:
   a. Collecting pending transactions into a block.
   b. Performing a computational puzzle (Proof-of-Work) to find a valid hash.
   c. Adding the new block to the blockchain.
   d. Broadcasting it to all connected peers.
   Miners are rewarded with cryptocurrency for successfully mining a block.
3. **Multi-Node Blockchain Network**
   In this lab, we simulate three independent blockchain nodes (5001, 5002, 5003). Each node:
   a. Runs on a separate port.
   b. Maintains its own copy of the blockchain.
   c. Can connect with peers to share and validate blocks.
4. **Consensus Mechanism**
   We use the Longest Chain Rule: If multiple versions of the chain exist, the longest valid chain is chosen. This ensures all nodes agree on a single transaction history.
5. **Transactions & Mining Reward**
   Each transaction has:
   a. Sender
   b. Receiver
   c. Amount
   When mining a block:
   a. Pending transactions are added to the block.
   b. A reward transaction is added automatically to pay the miner.
6. **Chain Replacement**
   When /replace_chain is called:
   a. Node requests chains from peers.
   b. If it finds a longer and valid chain, it replaces its own. This keeps the blockchain consistent across all nodes.

## Code:

```
import datetime import hashlib
import json
from uuid import uuid4
from
urllib.parse
import rlparse
import
requests
```

```
from flask import Flask, jsonify, request

class Blockchain:
    def _init_(self):
        self.chain = [] self.transactions = []
        self.create_block(proof=1,
previous_hash='0')
        self.nodes = set()

    def create_block(self, proof, previous_hash):
        block = {'index': len(self.chain) + 1,
'timestamp': str(datetime.datetime.now()), 'proof':
proof, 'previous_hash': previous_hash,
'transactions': self.transactions}
        self.transactions = []
        self.chain.append(block) return
        block

    def get_previous_block(self): return
        self.chain[-1]

    def proof_of_work(self, previous_proof):
        new_proof = 1
        while True: hash_operation =
hashlib.sha256(str(new_proof ** 2 -
previous_proof ** 2).encode()).hexdigest()
            if hash_operation.startswith('0000'): return
                new_proof
            new_proof += 1

    def hash(self, block):
        encoded_block = json.dumps(block,
sort_keys=True).encode()
        return
hashlib.sha256(encoded_block).hexdigest()

    def is_chain_valid(self, chain): if not
        chain:
            return False previous_block =
        chain[0] block_index = 1
        while block_index < len(chain): block =
            chain[block_index]
            if block['previous_hash'] !=
self.hash(previous_block):
                return False
            previous_proof =
            previous_block['proof'] proof =
            block['proof']
            hash_operation =
hashlib.sha256(str(proof ** 2 -
previous_proof
** 2).encode()).hexdigest()
            if not
                hash_operation.startswith('0000
                '): return False
            previous_block = block
            block_index += 1
        return True

    def add_transaction(self, sender,
receiver, amount):
        transaction = {'sender': sender,
'receiver': receiver, 'amount': amount}
        self.transactions.append(transaction)
        previous_block =
        self.get_previous_block() return
        previous_block['index'] + 1

    def add_node(self,
        address): parsed_url =
        urlparse(address) if
        parsed_url.netloc:
            self.nodes.add(parsed_url.netloc)

    def replace_chain(self):
        network = self.nodes
        longest_chain = None
        max_length =
        len(self.chain) for node
        in network:
            try:
                response =
requests.get(f'http://{node}/get_chain',
timeout=5)
                if response.status_code ==
                    200: data = response.json()
                    length =
                    data['length']
                    chain =
                    data['chain']
                    if length > max_length and
self.is_chain_valid(chain):
                        max_length = length
                        longest_chain = chain
            except requests.RequestException:
                continue
        if longest_chain is not
```

```
        None:   self.chain   =
        longest_chain
        return True
    return False
app = Flask(_name_)
node_address = str(uuid4()).replace('-', '')
blockchain = Blockchain()

@app.route('/mine_block', methods=['GET']) def
mine_block():
    previous_block = blockchain.get_previous_block()
    previous_proof = previous_block['proof']
    proof =
blockchain.proof_of_work(previous_proof)
    previous_hash =
blockchain.hash(previous_block)

blockchain.add_transaction(sender=node_addr ess,
receiver='Abcde', amount=1)
    block = blockchain.create_block(proof,
previous_hash)
    response = {'message': 'Congratulations, you just
mined a block!', 'index': block['index'], 'timestamp':
block['timestamp'], 'proof': block['proof'],
'previous_hash': block['previous_hash'],
'transactions': block['transactions']}
    return jsonify(response), 200

@app.route('/get_chain', methods=['GET']) def
get_chain():
    response = {'chain': blockchain.chain, 'length':
len(blockchain.chain)}
    return jsonify(response), 200

@app.route('/is_valid', methods=['GET']) def
is_valid():
    valid =
blockchain.is_chain_valid(blockchain.chain)
    if valid:
        response = {'message': 'All good. The
Blockchain is valid.'}
    else:
        response = {'message': 'Houston, we have a
problem. The Blockchain is not valid.'}
    return jsonify(response), 200

@app.route('/add_transaction', methods=['POST'])
def add_transaction():
    transaction_data = request.get_json()
    if not transaction_data:
        return jsonify({'message': 'Request
body must contain JSON'}), 400
    required_keys = ['sender',
'receiver', 'amount']
    if not all(key in transaction_data for key
in required_keys):
        return jsonify({'message': 'Some
elements of the transaction are missing'}),
400
    index =
blockchain.add_transaction(transaction_da
ta['s ender'], transaction_data['receiver'],
transaction_data['amount'])
    response = {'message': f'This transaction
will be added to Block {index}'}
    return jsonify(response), 201

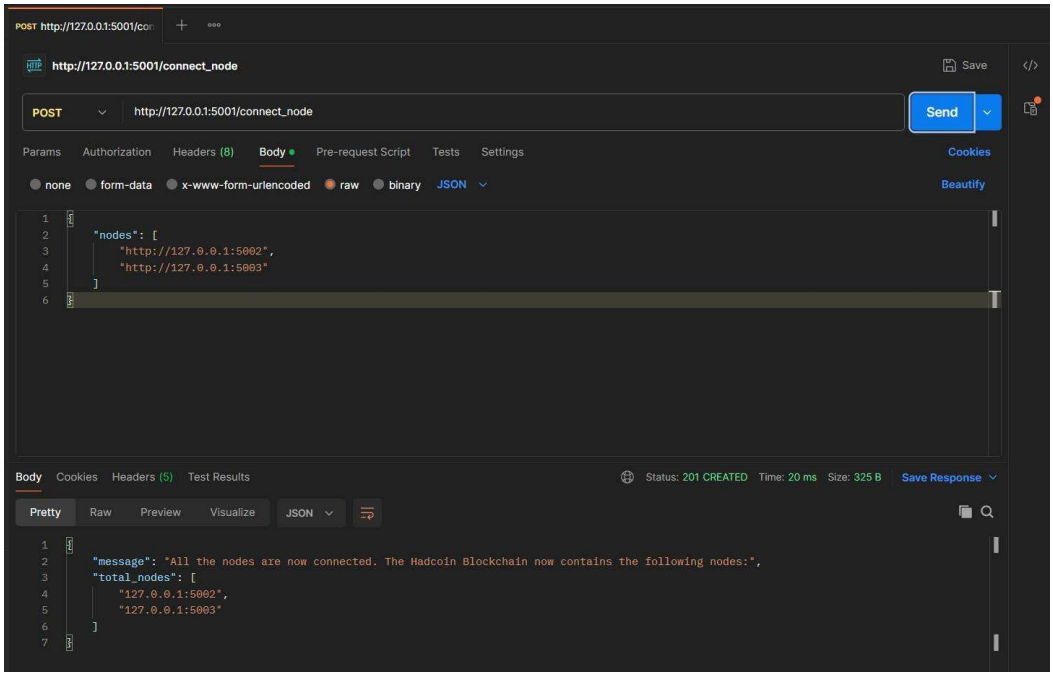
@app.route('/connect_node',
methods=['POST'])
def connect_node():
    data =
    request.get_json() if
    not data:
        return jsonify({'message': 'Request
body must contain JSON'}), 400
    nodes =
    data.get('nodes') if
    not nodes:
        return jsonify({'message': 'No node'}),
    400 for node in nodes:
        blockchain.add_node(node)
    response = {'message': 'All the nodes are
now connected. The Hadcoin Blockchain
now contains the following nodes:',
'total_nodes': list(blockchain.nodes)}
    return jsonify(response), 201

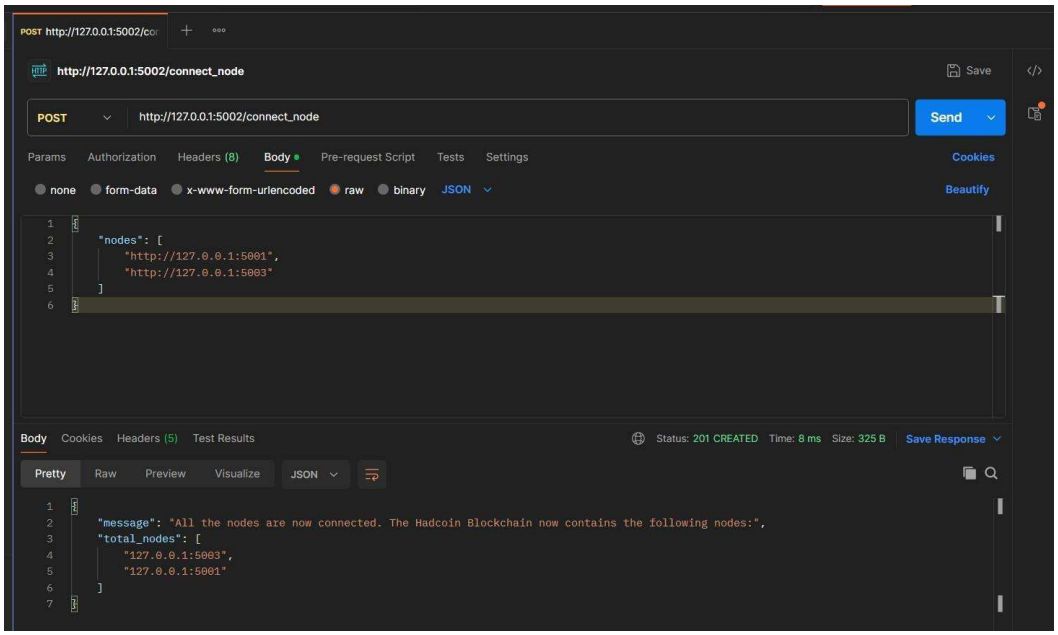
@app.route('/replace_chain',
methods=['GET']) def replace_chain():
    is_chain_replaced =
blockchain.replace_chain()
    if is_chain_replaced:
        response = {'message': 'The nodes had
different chains so the chain was replaced
by the longest one.', 'new_chain':
blockchain.chain}
    else:
        response = {'message': 'All good. The
chain is the largest one.', 'actual_chain':
blockchain.chain}
```

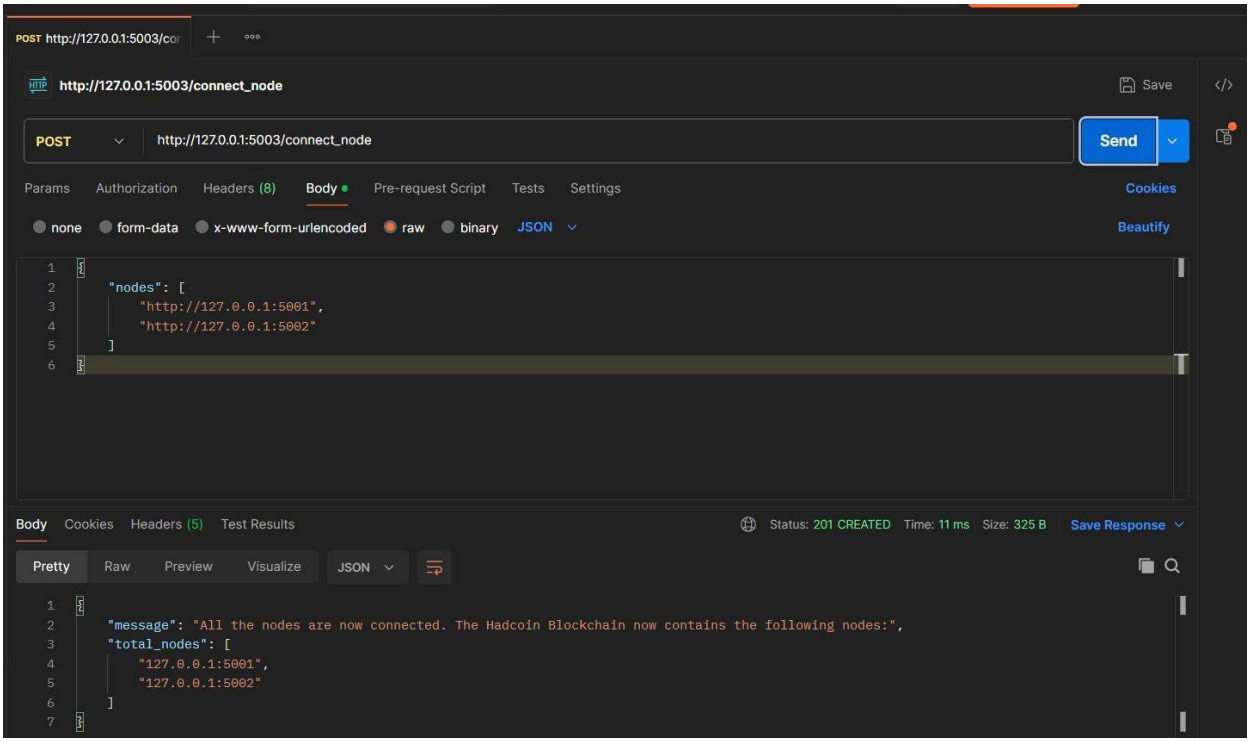
```
    return jsonify(response), 200 if __name__ == '__main__':
        app.run(host='0.0.0.0', port=5001, debug=False)
```

## Output:

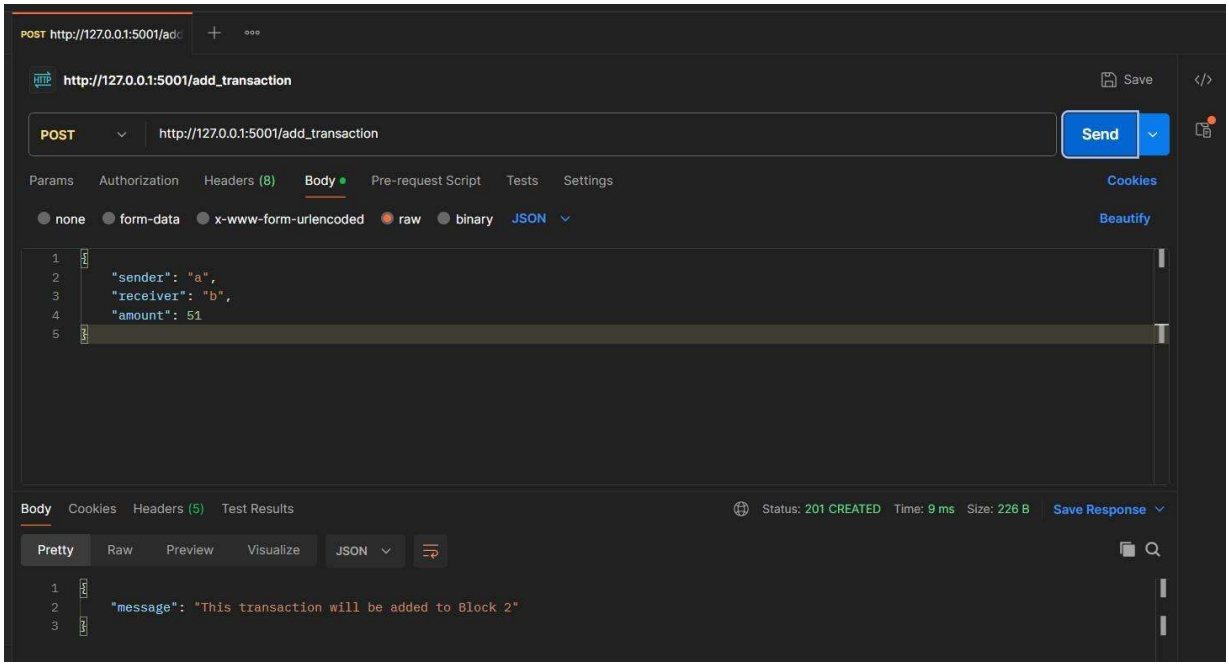
Connecting Nodes Using Postman:

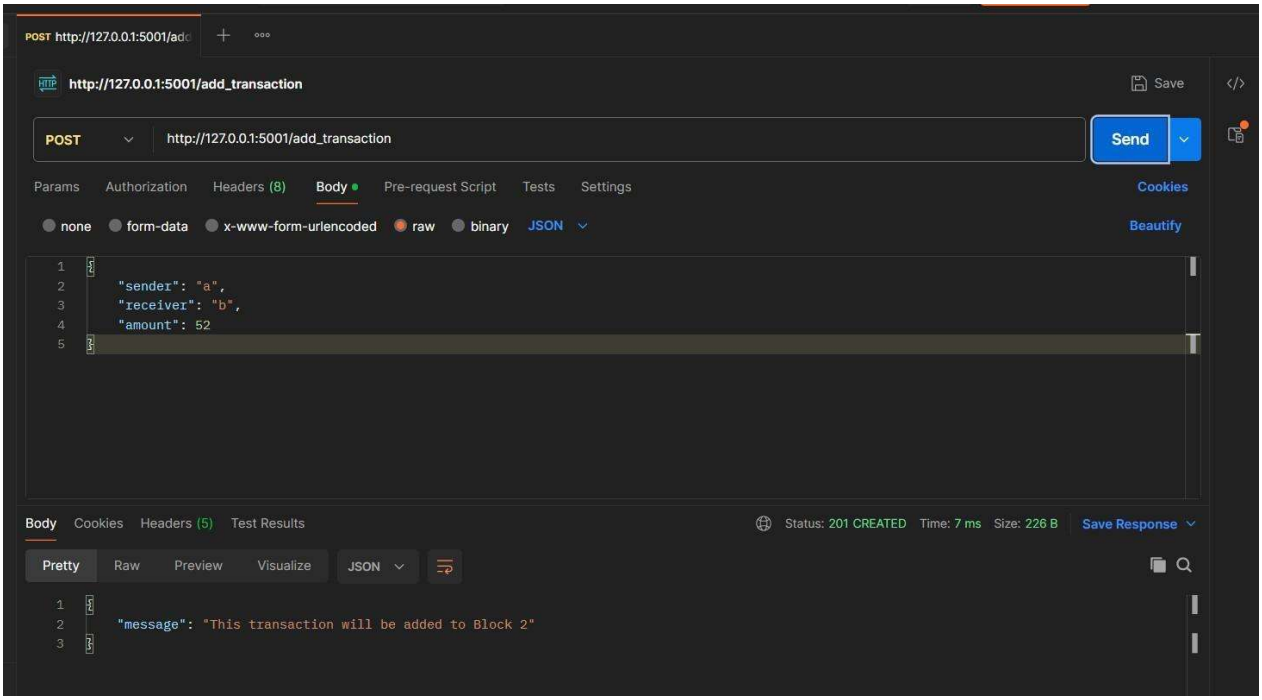


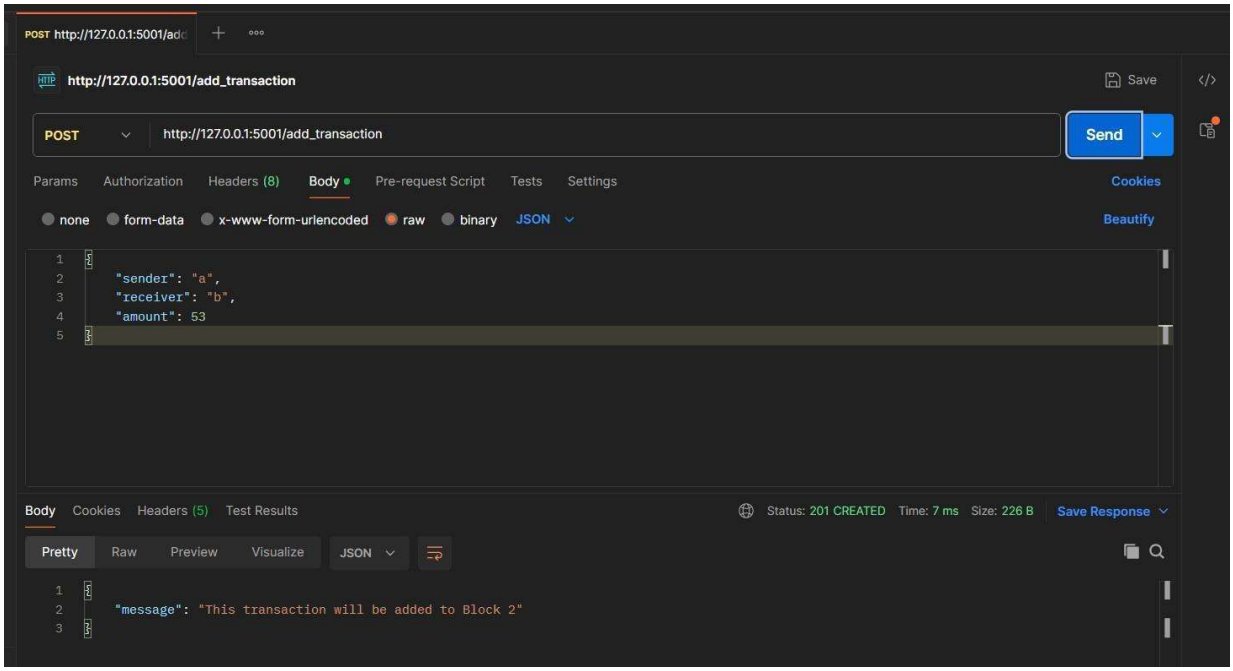





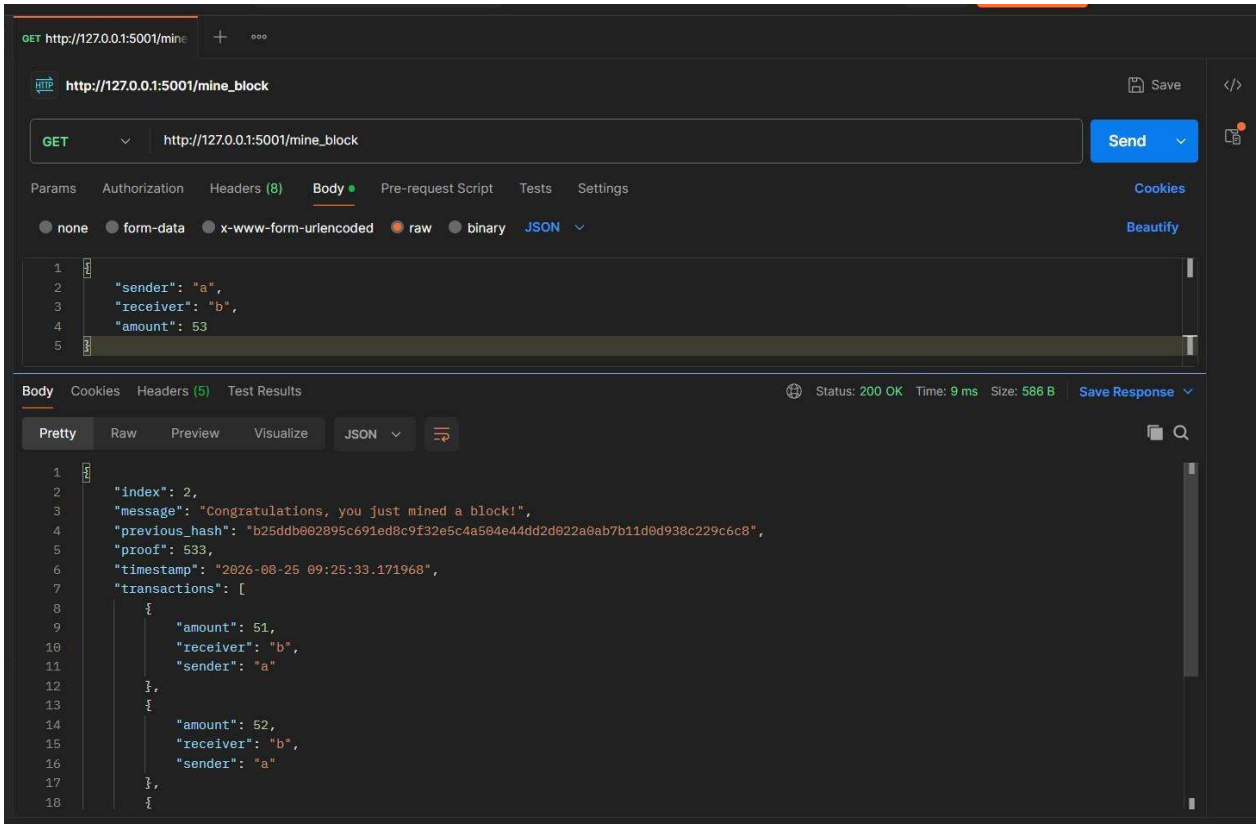

Add 3 transactions:




POST http://127.0.0.1:5001/add_transaction
http://127.0.0.1:5001/add_transaction
Save
POST
http://127.0.0.1:5001/add_transaction
Send
Params Authorization Headers (8) Body Pre-request Script Tests Settings
Cookies
none form-data x-www-form-urlencoded raw binary JSON
Beautify
{
"sender": "a",
"receiver": "b",
"amount": 52
}
Body Cookies Headers (5) Test Results
Status: 201 CREATED Time: 7 ms Size: 226 B
Save Response
Pretty Raw Preview Visualize JSON
{
"message": "This transaction will be added to Block 2"
}

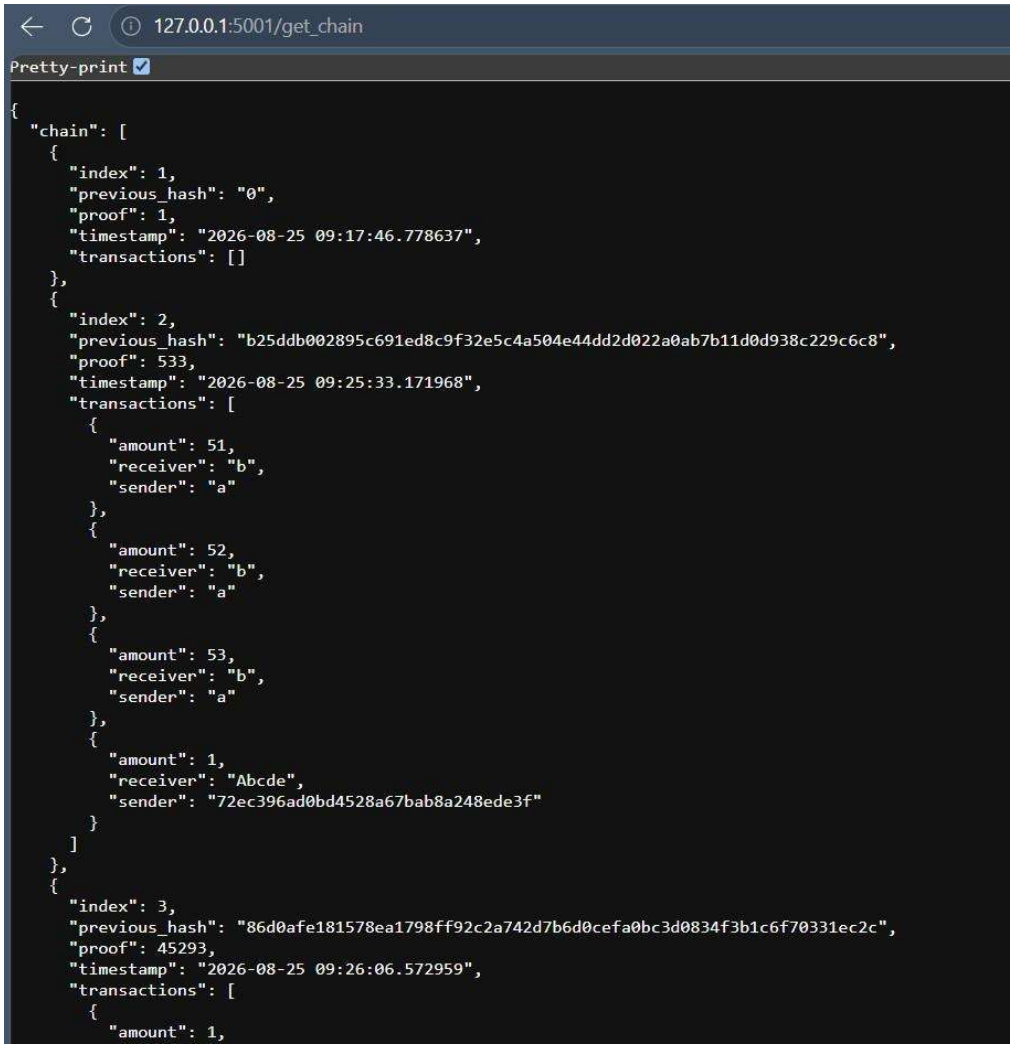

POST http://127.0.0.1:5001/add_transaction
http://127.0.0.1:5001/add_transaction
Save
POST
http://127.0.0.1:5001/add_transaction
Send
Params Authorization Headers (8) Body Pre-request Script Tests Settings
Cookies
none form-data x-www-form-urlencoded raw binary JSON
Beautify
{
"sender": "a",
"receiver": "b",
"amount": 53
}
Body Cookies Headers (5) Test Results
Status: 201 CREATED Time: 7 ms Size: 226 B
Save Response
Pretty Raw Preview Visualize JSON
{
"message": "This transaction will be added to Block 2"
}

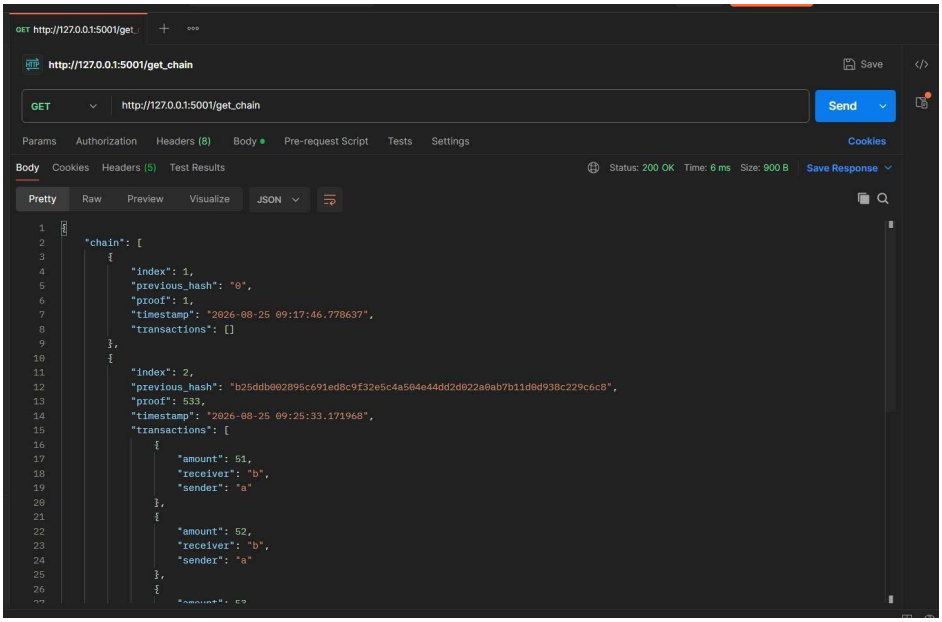
Mine Block:

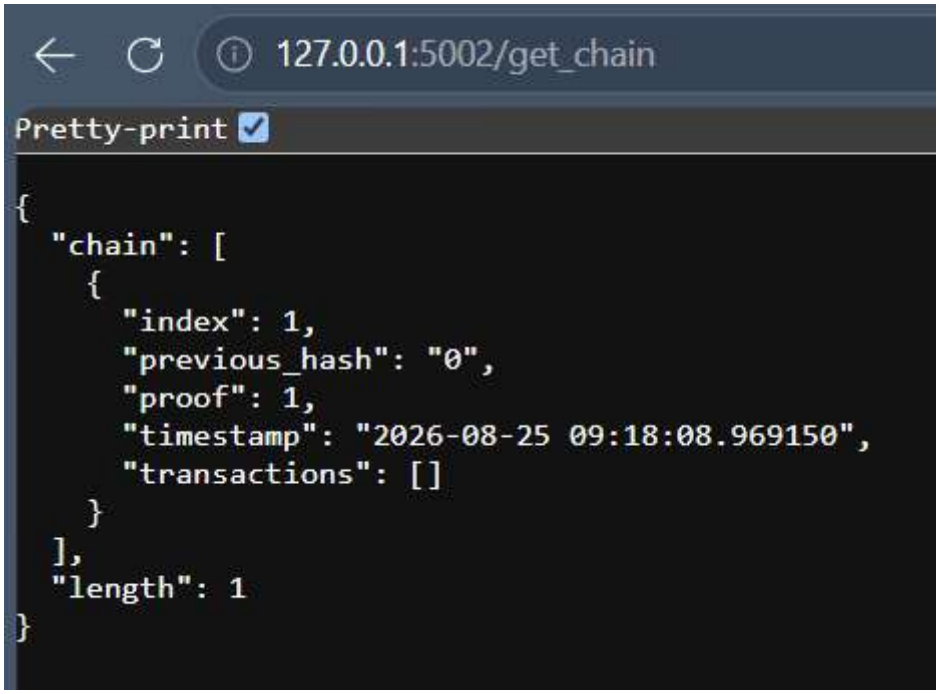

GET http://127.0.0.1:5001/mine
http://127.0.0.1:5001/mine_block
Save
GET
http://127.0.0.1:5001/mine_block
Send
Params
Authorization
Headers (8)
Body
Pre-request Script
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
JSON
Beautify
{
"sender": "a",
"receiver": "b",
"amount": 53
}
Body
Cookies
Headers (5)
Test Results
Status: 200 OK
Time: 9 ms
Size: 586 B
Save Response
Pretty
Raw
Preview
Visualize
JSON
{
"index": 2,
"message": "Congratulations, you just mined a block!",
"previous_hash": "b25ddb002895c691ed8c9f32e5c4a504e44dd2d022a0ab7b11d0d938c229c6c8",
"proof": 533,
"timestamp": "2026-08-25 09:25:33.171968",
"transactions": [
{
"amount": 51,
"receiver": "b",
"sender": "a"
},
{
"amount": 52,
"receiver": "b",
"sender": "a"
},
{

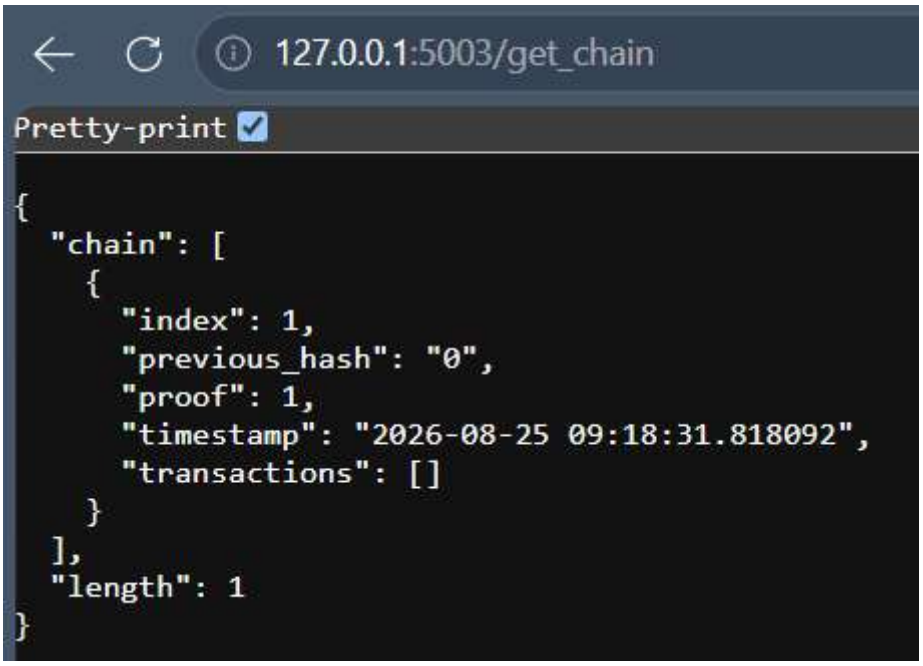

127.0.0.1:5001/mine_block
Pretty-print
{
"index": 3,
"message": "Congratulations, you just mined a block!",
"previous_hash": "86d0afe181578ea1798ff92c2a742d7b6d0cefa0bc3d0834f3b1c6f70331ec2c",
"proof": 45293,
"timestamp": "2026-08-25 09:26:06.572959",
"transactions": [
{
"amount": 1,
"receiver": "Abcde",
"sender": "72ec396ad0bd4528a67bab8a248ede3f"
}
]
}

Check the Chains:

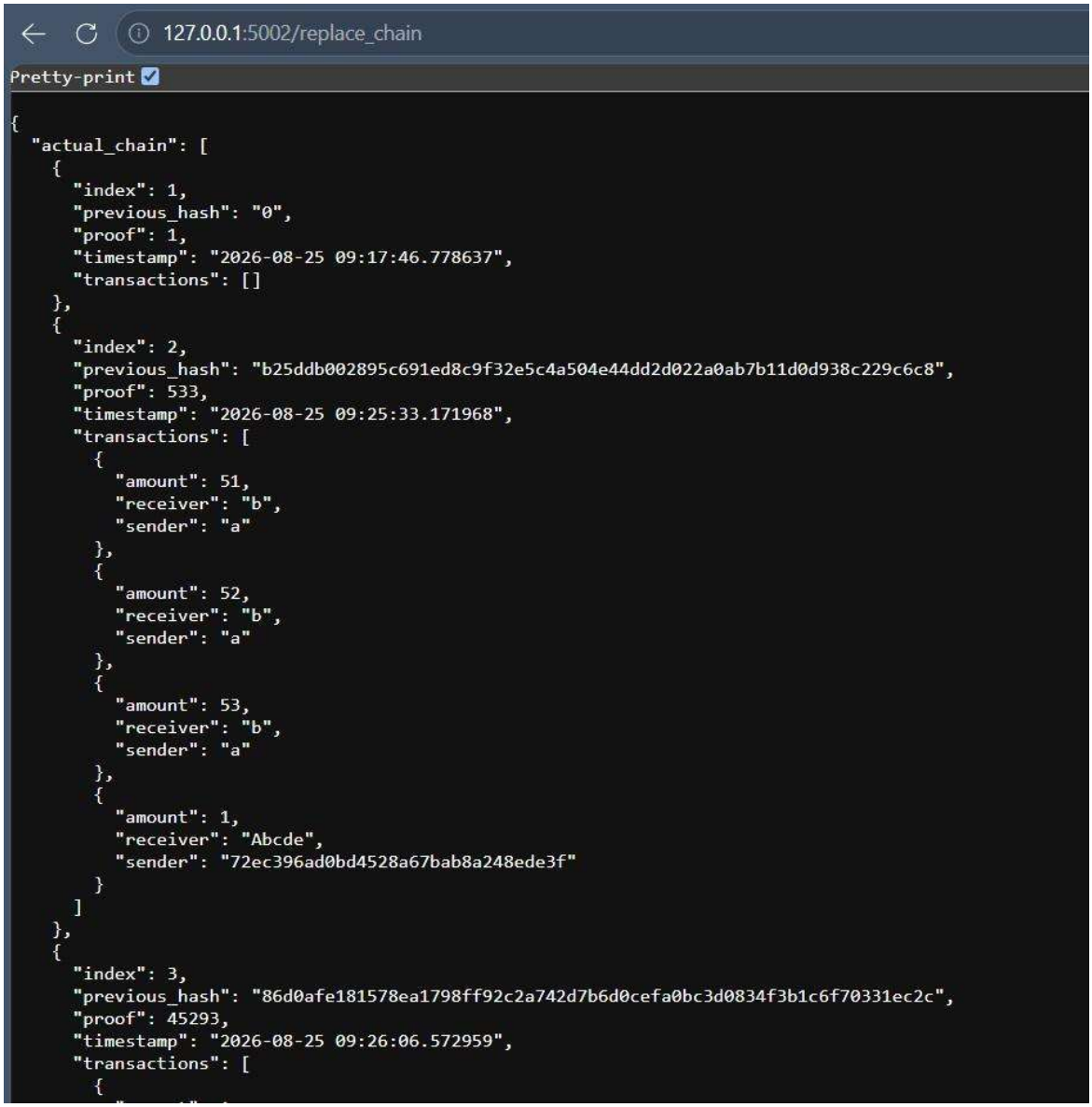



127.0.0.1:5002/get_chain
Pretty-print
{
"chain": [
{
"index": 1,
"previous_hash": "0",
"proof": 1,
"timestamp": "2026-08-25 09:18:08.969150",
"transactions": []
}
],
"length": 1
}


127.0.0.1:5003/get_chain
Pretty-print
{
"chain": [
{
"index": 1,
"previous_hash": "0",
"proof": 1,
"timestamp": "2026-08-25 09:18:31.818092",
"transactions": []
}
],
"length": 1
}

Demonstrate consensus:



127.0.0.1:5003/replace_chain

Pretty-print


```
{
  "message": "The nodes had different chains so the chain was replaced by the longest one.",
  "new_chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": "2026-08-25 09:17:46.778637",
      "transactions": []
    },
    {
      "index": 2,
      "previous_hash": "b25ddb002895c691ed8c9f32e5c4a504e44dd2d022a0ab7b11d0d938c229c6c8",
      "proof": 533,
      "timestamp": "2026-08-25 09:25:33.171968",
      "transactions": [
        {
          "amount": 51,
          "receiver": "b",
          "sender": "a"
        },
        {
          "amount": 52,
          "receiver": "b",
          "sender": "a"
        },
        {
          "amount": 53,
          "receiver": "b",
          "sender": "a"
        },
        {
          "amount": 1,
          "receiver": "Abcde",
          "sender": "72ec396ad0bd4528a67bab8a248ede3f"
        }
      ]
    },
    {
      "index": 3,
      "previous_hash": "86d0afe181578ea1798ff92c2a742d7b6d0cefa0bc3d0834f3b1c6f70331ec2c",
      "proof": 45293,
      "timestamp": "2026-08-25 09:26:06.572959",
      "transactions": [
        {
```


Validate all 3 blocks:

127.0.0.1:5001/is_valid

Pretty-print


```
{
  "message": "All good. The Blockchain is valid."
}
```


127.0.0.1:5002/is_valid

Pretty-print


```
{
  "message": "All good. The Blockchain is valid."
}
```


127.0.0.1:5003/is_valid

Pretty-print


```
{
  "message": "All good. The Blockchain is valid."
}
```

## Conclusion:

We developed a cryptocurrency using Python and implemented mining in a simulated three-node blockchain network. Each node maintained its own ledger and synchronized with peers using the Longest Chain Rule to ensure consistency. Mining was performed through Proof-of-Work, securely adding transactions and rewarding miners. This demonstrated key blockchain concepts, including decentralized consensus, transaction validation, and network synchronization.